4.8. Автоматизация ИБ-проверок

4. Python в задачах информационной безопасности

В этом уроке

- Что такое автоматизация ИБ-проверок
- Зачем писать собственные скрипты
- Регулярные выражения для поиска ИБ-рисков
- Условия и циклы для автоматической логики
- Автоматическая проверка данных
- Формирование отчётов
- Написание простого «проверяльщика»

Что такое ИБ-проверки

🛡️ Информационная безопасность (ИБ) требует регулярного анализа систем, чтобы обнаруживать потенциальные угрозы.

Такие проверки можно проводить вручную, но автоматизация значительно упрощает процесс.

Основные направления автоматических проверок:

Логи: анализ попыток входа, ошибок, аномалий.

Конфигурации: поиск уязвимых или небезопасных настроек.

Входные данные: проверка пользовательских данных на корректность и безопасность.

Роль автоматизации — это:

Повышение скорости и регулярности аудита.

Исключение человеческого фактора.

Масштабируемость при работе с большими объёмами данных.

Примеры задач для автоматизации

- Проверка паролей на слабость:
  - Примеры слабых шаблонов: 123456, admin, password, дата рождения и т.д.
- Сканирование портов:
  - Выявление открытых портов и уязвимых сервисов.
  - Проверка наличия открытого SSH, Telnet, FTP и др.
- Анализ логов на XSS и SQL-инъекции:
  - Поиск подозрительных конструкций вроде <script>, SELECT, DROP.
- Поиск подозрительных IP-адресов:
  - Проверка входов с нетипичных стран.
  - Частые неудачные попытки входа с одного IP.
  - Использование IP-блоклистов.
- Такие задачи можно объединить в один скрипт и запускать по расписанию (cron, Task Scheduler).

Компоненты скриптовой проверки

- Ввод данных:
  - Считывание логов, конфигураций, текста из файлов или строки.
- Фильтрация условий:
  - С помощью if, in, not in, startswith, endswith.
- Поиск паттернов:
  - С применением регулярных выражений (модуль re).
- Обработка в цикле:
  - Проход по строкам файла, анализ каждой строки по правилам. Формирование отчета:
- Вывод результатов в консоль, сохранение в текст/JSON/CSV.
- 📌 Это скелет любого аудиторского или защитного скрипта.

Регулярные выражения в ИБ

Регулярные выражения в ИБ используются для:

- Поиска IP-адресов: 192.168.0.1, 10.0.0.254
- Нахождения email-адресов
- Определения XSS (<script>) и SQL-инъекций (SELECT, DROP, ' OR 1=1)
- Детектирования шаблонов в логах
- Валидации пользовательского ввода

```
# Найти IP-адреса
re.findall(r"\b(?:\d{1,3}\.){3}\d{1,3}\b", text)

# Найти XSS или SQL-инъекции
re.findall(r"(<script>|SELECT|DROP|INSERT|--)", line, re.IGNORECASE)
```

Использование циклов и условий

Циклы и условия — основа логики автоматических проверок.

```
for line in open("auth.log"):
    if "admin" in line and "fail" in line:
        print("Найдено подозрительное действие:", line.strip())
```

Что делает этот код:

- Проходит по всем строкам лога.
- Если в строке есть "admin" и "fail" — выводит её как потенциально опасную.

✅ Применение:

- Анализ логов авторизации.
- Выявление частых срабатываний тревог.
- Поиск совпадений по нескольким критериям.

Проверка входных данных

Безопасность пользовательских данных критична.

Автоматическая проверка позволяет избежать атак типа:

- XSS (ввод HTML-кода)
- SQL-инъекции
- Подделка email или телефонов

```python
import re


email = "test@example.com"
if not re.match(r"[^@]+@[^@]+\.[^@]+", email):
    print("Невалидный email!")
```

Можно проверять:

- Пароли: длина, спецсимволы, цифры.
- Телефоны: формат +7(XXX)XXX-XX-XX.
- Имена: только буквы, отсутствие спецсимволов.

Формирование отчета о проверке

Цели отчета:

- Зафиксировать найденные инциденты.
- Упростить анализ результатов.
- Передать информацию другим сотрудникам.

```
with open("report.txt", "w") as f:
    for alert in alerts:
        f.write(alert + "\n")
```

📌 Форматы отчетов:

- txt — простой текст.
- csv — удобно открывать в Excel.
- json — структурированный, пригоден для автоматической обработки

Утилита-проверяльщик

```python
import re

with open("web_access.log") as f:
    for line in f:
        if re.search(r"(SELECT|DROP|INSERT|OR 1=1)", line, re.IGNORECASE):
            print("[!] Подозрительная строка:", line.strip())
```

Что делает скрипт:

- Читает файл построчно.
- Проверяет каждую строку на ключевые слова SQL-инъекций.
- Выводит подозрительные строки.

Анализ логов SSH

Этот код:

- Ищет строки с ошибками входа.
- Извлекает IP.
- Считает количество неудачных попыток по IP.
- Выводит только подозрительных.

```
fail_ips = {}

with open("auth.log") as f:
    for line in f:
        if "Failed password" in line:
            ip = re.findall(r"\d+\.\d+\.\d+\.\d+", line)
            if ip:
                ip = ip[0]
                fail_ips[ip] = fail_ips.get(ip, 0) + 1

for ip, count in fail_ips.items():
    if count > 5:
        print(f"[!] {ip} — {count} попыток входа")
```

📌 Можно дополнительно блокировать такие IP с помощью iptables или fail2ban

Автоматизация через планировщик

Чтобы скрипты запускались автоматически, используют системные планировщики:

📌 В Linux: cron — стандартный планировщик.

```
0 * * * * python3 /home/user/check_logs.py >> /home/user/report.log
```

В Windows — аналог: Task Scheduler.

Автоматические реакции на угрозы

Кроме обнаружения угроз, возможна автоматическая реакция:

📌 Примеры:

- 🚫 Блокировка IP-адреса.
- 🔒 Отключение пользователя.
- 📬 Отправка email-уведомления.
- 🗃️ Создание тикета в системе заявок (Jira, GLPI и др).

```
import os
os.system("iptables -A INPUT -s 192.168.1.10 -j DROP")
```

Оптимизация и масштабирование

- При росте количества логов или проверок скрипты могут замедляться.
- 📌 Рекомендации:
  - Использовать модули парсинга (pandas, csv, json).
  - Разделить скрипт на модули (анализ, отчёт, реакция).
  - Хранить данные во временных SQLite-файлах.
  - Писать лог-файлы ошибок и истории событий.
- 📌 Для больших проектов:
  - Подключение к системам SIEM.
  - Отправка логов на удалённый сервер.
  - Контейнеризация (Docker) скриптов.

Итоги урока

В рамках урока мы изучили:

- Использование регулярных выражений для поиска уязвимостей.
- Циклы и условия для анализа логов и данных.
- Формирование отчетов и запуск по расписанию (cron).
- Создание утилиты-проверяльщика.
- Возможность автоматической реакции на угрозы.
- Подходы к масштабированию и оптимизации.